Implementation Companion

Python Examples for Cyber Control, Differential Games, RL, and MADRL

From mathematical formulation to reproducible Note 1 experiments

Companion to Note 1

Contents

1. Purpose

This companion explains the Note 1 Python files. It maps equations to runnable code and points to extension hooks used in research prototypes. PINN/PIDL examples are handled in the separate Note 2 repository.

2. File Map

File	Role
`shared_setup.py`	local helper that finds the shared foundation package in a sibling workspace
`cyber_dynamics.py`	compatibility wrapper for shared `cybercontrol` ODE and RK4 utilities
`sampled_continuous_impulse_env.py`	sampled-data environment with continuous flow, jump map, and parameterized actions
`fbsm_malware_baseline.py`	open-loop FBSM baseline from PMP
`ddqn_cyber_defense.py`	discrete-action DDQN defender against a scripted attacker
`madrl_ctde_parameterized_game.py`	compact CTDE attacker-defender baseline
`node_level_robustness.py`	node-level epidemic-model robustness experiment

3. General Implementation Architecture

A clean implementation should separate four layers:

1. model layer: ODE right-hand side, parameters, constraints;
2. environment layer: reset, action decoding, jump map, ODE integration, reward;
3. algorithm layer: FBSM, DDQN, and MADRL;
4. evaluation layer: baselines, robustness, plots, logs, seeds.

The code folder follows this structure. The environment is intentionally independent of DQN, PPO, or MADRL so that different learning algorithms can reuse the same cyber model.

4. ODE, Sampled Flow, and Impulse Environment

The central environment step is

$$\boxed{s_k} \to \boxed{a_D^k, a_A^k} \to \boxed{x_k^+ = G(x_k, a_k)} \to \boxed{x_{k+1} = \mathrm{RK4}(f, x_k^+)} \to \boxed{r_D^k, r_A^k}. \tag{1}$$

The implementation is in `sampled_continuous_impulse_env.py`. The most important design choice is to keep jump effects separate from continuous flow effects.

Listing 1: Core idea of a sampled-data flow and impulse environment step.

```
def step(self, defender_action, attacker_action):
```

```
    dpar = self.defense_parameters(defender_action)
    apar = self.attack_parameters(attacker_action)
    pre_jump = self.state.copy()
    post_jump = self.jump_map(pre_jump, dpar)
    rhs = lambda x, t: sampled_sir_flow_rhs(x, dpar, apar, self.cfg.params)
    next_state, path = rk4_integrate(rhs, post_jump, t0=self.t*self.cfg.dt,
                                     dt=self.cfg.dt, substeps=self.cfg.substeps,
                                     project=project_simplex3)
    reward = compute_reward(path, dpar, apar)
    return next_state, reward, done, info
```

4.1 How to extend this environment

To extend from the simple $[S, I, R]$ model to a degree-based network model, replace the state vector with

$$x = [S_1, I_1, R_1, \ldots, S_K, I_K, R_K] \tag{2}$$

where K is the number of degree classes. Then update the RHS so that infection pressure for degree class k uses a degree-weighted infected neighbor probability. Deception remains an action-dependent reduction of effective infection pressure during a decision interval rather than a core state compartment. The RL environment interface does not change.

5. FBSM Baseline

The FBSM file solves a continuous-control malware problem. It should be used as a classical baseline when the model is known and the control is open-loop. It is not a substitute for DDQN or MADRL because it does not learn a feedback policy.

Listing 2: FBSM update logic.

```
for iteration in range(max_iter):
    # forward state integration using current u(t)
    for k in range(n):
        x[k+1] = rk4_state_step(x[k], u[k], h, beta, gamma)
    # backward costate integration from terminal condition
    lambda[n] = terminal_gradient
    for k in range(n, 0, -1):
        lambda[k-1] = rk4_costate_step_backward(x[k], lambda[k], u[k], h)
    # stationarity update and relaxation
    u_new[k] = clip(S[k]*(lambda_S[k]-lambda_R[k])/B, 0, u_max)
    u = relax*u_new + (1-relax)*u
```

6. DDQN Example

The DDQN example trains the defender with discrete actions. The attacker is scripted so that the defender first learns against a stable environment. This is the recommended first step before full MADRL.

DDQN workflow.

1. initialize Q-network and target Q-network;
2. collect transitions (s, a, r, s', d) using epsilon-greedy exploration;

3. store transitions in replay buffer;
4. sample mini-batches;
5. compute DDQN target;
6. update the online network;
7. periodically copy online weights to target weights;
8. validate against static and rule-based policies.

7. MADRL / CTDE Example

The MADRL file uses decentralized actors and centralized critics. It is intentionally compact. The defender and attacker both receive the same state observation in the example, but this can be changed to partial observations.

7.1 General extension to richer MADRL

For research runs, add:

- generalized advantage estimation (GAE);
- clipped PPO ratios for MAPPO;
- opponent pools to reduce cycling;
- multiple seeds and cross-play matrices;
- best-response retraining as an approximate exploitability test;
- separate observation functions for attacker and defender.

8. Evaluation Scripts to Add in Future Work

The current scripts print losses and returns. A research version should save:

- CSV trajectories for S, I, R together with action, reward, and baseline diagnostics;
- training curves;
- baseline comparison tables;
- robustness results across parameter perturbations;
- policy action distributions over time.

9. Complex Extensions

9.1 Degree-based HODGE-style model

To implement a degree-based propaganda model, let the state contain P_k and C_k for each degree k. The force of influence uses

$$\Theta_P(t) = \frac{\sum_k k p_k P_k(t)}{\sum_k k p_k}, \qquad \Theta_C(t) = \frac{\sum_k k p_k C_k(t)}{\sum_k k p_k}. \tag{3}$$

Continuous propaganda investment changes ODE rates, while impulsive counter-propaganda applies a jump map at selected epochs. The same environment structure still applies.

9.2 Connection to Note 2

If the research question becomes inverse learning, missing-mechanism learning, direct neural control, or PMP-informed PINNs, continue in the Note 2 repository. That repository contains the PINN/PIDL code, loss terms, and neural-network checks. Note 1 should remain focused on continuous-control baselines, sampled-data MDPs, and attacker-defender Markov games.

10. Suggested Project Sequence

1. Run FBSM and DDQN on the same simple malware model.
2. Add parameterized actions and compare no defense, static patch, rule-based, DDQN.
3. Add attacker learning and compare independent learning with CTDE.
4. Add node-level graph experiments and parameter-mismatch robustness.
5. Move to Note 2 if the next step is inverse PINN, PIDL, or PMP-informed neural control.
6. Extend to APT or misinformation dynamics after the simpler timing and baseline checks are stable.